UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Software

Asignatura: Ingeniería de Requerimientos (ISR-401)

TA-IND-01

IEEE 29148 y tipos de requisitos de un sistema conocido

Trabajo autónomo individual (actividad sumativa)

Estudiante: Vélez Vinces Jostin Gustavo

Cuarto nivel · Paralelo B

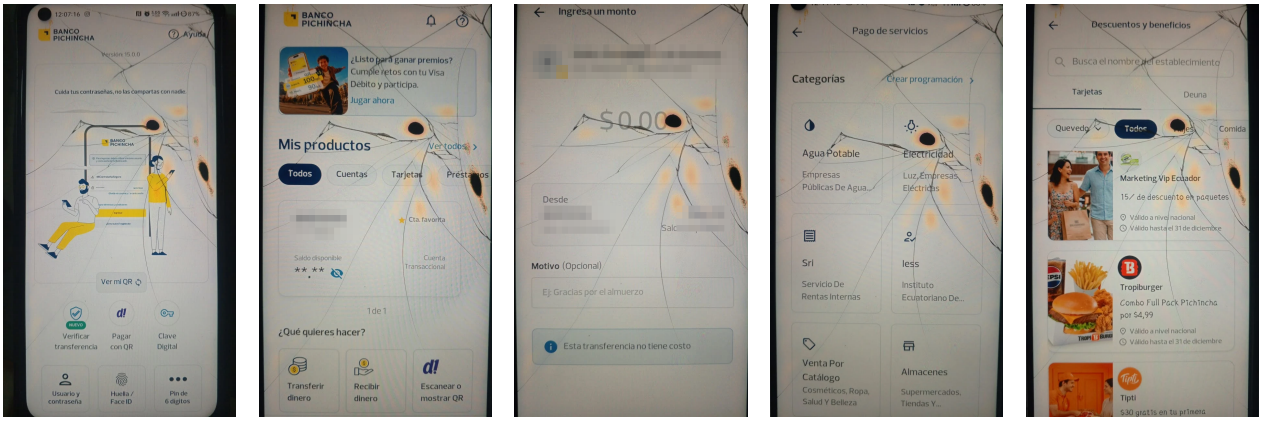
Docente: Ing. Gleiston Cicerón Guerrero Ulloa, PhD

Período académico 2026–2027 SPA

Quevedo, 11 de noviembre de 2026

1. Sistema elegido

La aplicación Banca Móvil del Banco Pichincha es una herramienta digital que permite a sus usuarios realizar operaciones bancarias desde sus teléfonos celulares sin necesidad de acudir a una agencia. A través de ella se pueden consultar saldos y movimientos, realizar transferencias y pagar servicios básicos (Figura 1). Está dirigida a los clientes del Banco Pichincha que necesitan administrar su dinero mediante herramientas digitales. En Ecuador, se utiliza desde cualquier lugar que cuente con conexión a internet, como el hogar, el trabajo o durante los desplazamientos, permitiendo realizar operaciones bancarias en diferentes momentos del día. De esta manera, facilita el acceso a los servicios bancarios y reduce la necesidad de acudir presencialmente a una agencia.



(a) Inicio de sesión. (b) Menú principal. (c) Transferencia. (d) Pago de servicios. (e) Descuentos y beneficios.

Figura 1: Pantallas de la aplicación de banca móvil de Banco Pichincha (versión 15.0.0). Fotografías propias tomadas con otro teléfono, porque la aplicación no permite capturas de pantalla; los datos personales están pixelados.

2. Resumen analítico de ISO/IEC/IEEE 29148:2018

2.1. Requisito bien formado

SWEBOK define un requisito de software como una propiedad que algo debe tener para resolver un problema del mundo real [1]. La norma lo define como un enunciado que expresa una necesidad junto con sus condiciones y restricciones (cláusula 5.2.4). Además, pide que los requisitos estén "bien formados", porque así es más fácil validarlos con los interesados y comprobar que reflejan lo que de verdad necesitan (cláusula 5.2.4).

Según la norma, un requisito bien formado presenta una o más de estas propiedades: lo debe cumplir el sistema para resolver un problema o lograr un objetivo; tiene condiciones medibles; está limitado por restricciones; describe lo que hace el sistema y no lo que hace el usuario; y se puede verificar (cláusula 5.2.4).

Para escribirlo, la norma pide un sujeto (el sistema), un verbo y la acción o restricción. "Deberá" (*shall*) se usa solo para lo obligatorio, en voz activa y en forma positiva, y se evita "deberá ser capaz de" (cláusula 5.2.4). La norma también diferencia las condiciones, que son atributos medibles del requisito, de las restricciones, que limitan el diseño, como una ley, el presupuesto o una plataforma que ya existe (cláusula 5.2.4).

Ejemplo (pantalla "Ingresa un monto" de la aplicación, Figura 1c): "La aplicación deberá mostrar el aviso 'Esta transferencia no tiene costo' antes de que el cliente confirme una transferencia sin costo". El sujeto es la aplicación, la acción es obligatoria, tiene una condición y se puede comprobar viendo la pantalla.

2.2. Características de un requisito individual

La norma dice que cada requisito, por sí solo, debe cumplir nueve características (cláusula 5.2.5):

1. **Necesario** (*necessary*): si se quita, falta algo que ningún otro requisito cubre, y además sigue vigente (cláusula 5.2.5).

2. **Apropiado** (*appropriate*): tiene el nivel de detalle que corresponde y no impone diseño sin necesidad (cláusula 5.2.5). Ejemplo: "La aplicación deberá permitir el ingreso mediante huella dactilar" es apropiado. "La app deberá usar la librería biométrica X" no lo es, porque ya decide el diseño.

3. **Inequívoco** (*unambiguous*): se entiende de una sola forma (cláusula 5.2.5). Si un requisito dice "mostrar el saldo rápidamente", el programador y el banco pueden entender tiempos distintos.

4. **Completo** (*complete*): se entiende sin buscar información en otro lado (cláusula 5.2.5). "La app deberá permitir transferir" queda incompleto si no dice desde qué cuenta, aunque la pantalla de la aplicación sí tiene el campo "Desde" (Figura 1c).

5. **Singular** (*singular*): pide una sola capacidad o restricción (cláusula 5.2.5). Ejemplo: "La app permitirá ver el saldo, ocultarlo con el ícono del ojo y transferir dinero" se debe separar en tres requisitos de la pantalla "Mis productos" (Figura 1b).

6. **Factible** (*feasible*): se puede hacer con el costo, el tiempo y la tecnología disponibles (cláusula 5.2.5). Pedir que una transferencia llegue en 0 segundos no es factible.

7. **Verificable** (*verifiable*): se puede demostrar que se cumplió, y es más fácil cuando es medible (cláusula 5.2.5). SWEBOK también considera la verificabilidad una propiedad esencial de los requisitos [1]. Si un requisito no se puede probar, no sirve, porque nadie puede saber si se cumplió. Ejemplo: "La app deberá mostrar promociones relevantes" no es verificable. En cambio, "La tarjeta de una promoción en la pantalla 'Descuentos y beneficios' deberá mostrar: establecimiento, descuento, cobertura, fecha de vigencia" se puede revisar dato por dato (Figura 1e).

8. **Correcto** (*correct*): representa bien la necesidad de la que salió (cláusula 5.2.5). Si el banco quiere avisar que la transferencia es gratis y el requisito pide "mostrar el costo de la transferencia", el requisito no es correcto.

9. **Conforme** (*conforming*): sigue la plantilla y el estilo de redacción aprobados (cláusula 5.2.5). Por ejemplo, todos los requisitos de este trabajo usan "deberá" como verbo obligatorio, con un sujeto, una acción y una condición. Si un requisito estuviera escrito con otra estructura, por ejemplo con "debe" o en voz pasiva, no sería conforme.

2.3. Características de un conjunto de requisitos

La norma aclara que algunas cualidades se revisan en el conjunto y no en cada requisito por separado (cláusula 5.2.6):

- **Completo** (*complete*): el conjunto describe todo lo necesario y no deja pendientes "por definir" (TBD), "por especificar" (TBS) o "por resolver" (TBR) (cláusula 5.2.6).

- **Consistente** (*consistent*): los requisitos no se repiten ni se contradicen, usan las mismas unidades y cada término significa siempre lo mismo (cláusula 5.2.6). Ejemplo: en la aplicación, la función QR aparece como "Pagar con QR", "Ver mi QR" y "Escanear o mostrar QR" (Figuras 1a y 1b). La especificación tiene que definir cada término para no llamar de formas distintas a una misma función.

- **Factible** (*feasible*): se puede hacer el conjunto entero, y es asequible (cláusula 5.2.6).

- **Comprensible** (*comprehensible*): se entiende qué se espera y cómo se relaciona con el sistema mayor (cláusula 5.2.6).
- **Validable** (*able to be validated*): si se cumple el conjunto, se satisfacen de verdad las necesidades dentro de los límites técnicos, de costo y legales (cláusula 5.2.6).

La norma advierte que revisar esto a tiempo evita que los requisitos crezcan sin control (*requirements creep*) (cláusula 5.2.6).

2.4. Términos ambiguos que la norma prohíbe

La norma pide que los requisitos digan "qué" se necesita y no "cómo". También pide evitar los términos vagos, porque no dejan verificar o se pueden entender de varias formas (cláusula 5.2.7). Los tipos son:

- superlativos ("el mejor");
- lenguaje subjetivo ("fácil de usar");
- pronombres vagos ("esto", "eso");
- adverbios y adjetivos ambiguos ("casi siempre", "mínimo") y conectores como "y/o";
- términos abiertos ("pero no limitado a");
- comparativos ("mejor que");
- escapatorias ("si es posible", "según corresponda");
- términos de totalidad ("todos", "siempre", "nunca");
- referencias incompletas, sin versión ni fecha (cláusula 5.2.7).

Ejemplo (pantalla "Pago de servicios", Figura 1d): "La app deberá permitir pagar todos los servicios de forma fácil y, si es posible, programarlos" tiene un término de totalidad, lenguaje subjetivo y una escapatoria. Se corrige con dos requisitos: "La aplicación deberá mostrar las siguientes categorías: Agua Potable, Electricidad, SRI, IESS, Venta por Catálogo, Almacenes" y "La aplicación deberá permitir crear una programación de pago desde la opción 'Crear programación'". Por último, la norma pide documentar los supuestos en los atributos del requisito (cláusula 5.2.7).

3. Glosario

Tabla 1: Glosario de términos de la cláusula 3.1 de ISO/IEC/IEEE 29148:2018 [2].

N.º	Término	Cláusula	Definición parafraseada	Ejemplo en la banca móvil
1	Requisito (*requirement*)	3.1.19	Enunciado que expresa una necesidad junto con sus condiciones y restricciones. Siempre se refiere a un sistema, software o servicio.	"La aplicación deberá mostrar el aviso 'Esta transferencia no tiene costo' antes de que el cliente confirme una transferencia sin costo."
2	Interesado (*stakeholder*)	3.1.28	Persona u organización que tiene un derecho, un interés o un reclamo sobre el sistema. Incluye usuarios, desarrolladores, clientes, operadores y entes reguladores.	El cliente, Banco Pichincha, el equipo que desarrolla la aplicación, la Superintendencia de Bancos y los comercios que aparecen en "Descuentos y beneficios".

N.º	Término	Cláusula	Definición parafraseada	Ejemplo en la banca móvil
3	Usuario (*user*)	3.1.35	Persona o grupo que interactúa con el sistema o se beneficia de él mientras se usa. Es un tipo de interesado.	El cliente que entra con su huella y paga la luz en “Pago de servicios”. También quien recibe una transferencia se beneficia del sistema aunque no lo use en ese momento.
4	Condición (*condition*)	3.1.6	Atributo medible que indica en qué circunstancia o evento aplica un requisito.	En el requisito del aviso, la condición es “antes de que el cliente confirme la transferencia”.
5	Restricción (*constraint*)	3.1.7	Limitación impuesta desde fuera sobre el sistema, su diseño o el proceso con que se desarrolla. No la decide el equipo, se le impone.	La LOPDP obliga a la aplicación a tratar los datos personales del cliente según la ley, aunque al equipo le resultara más fácil hacerlo de otra forma.
6	Atributo (*attribute*)	3.1.2	Propiedad de algo que se puede distinguir por cantidad o por cualidad.	Los atributos de un requisito, como su ID (RQ-01), su tipo y su prioridad. En la aplicación, el “Saldo disponible” es un atributo de la cuenta.
7	Verificación (*verification*)	3.1.37	Confirmar con evidencia que se cumplió lo que estaba especificado. Responde a “¿lo construimos bien?”.	Un *tester* abre “Ingresa un monto” y comprueba que el aviso aparece con el texto exacto que pide el requisito.
8	Validación (*validation*)	3.1.36	Confirmar con evidencia que el sistema sirve para el uso previsto. Responde a “¿construimos el sistema correcto?”.	Se pide a clientes reales que paguen su planilla de agua desde la aplicación. Si no logran hacerlo sin ayuda, la aplicación no queda validada, aunque cumpla cada requisito escrito.
9	Trazabilidad de requisitos (*requirements traceability*)	3.1.23	Registrar de dónde viene un requisito (hacia arriba, el requisito “padre”) y en qué otros se divide (hacia abajo, los requisitos “hijos”).	La necesidad “pagar servicios desde el celular” (padre) se divide en RQ-05, que muestra las categorías, y RQ-06, que permite crear una programación de pago (hijos).
10	Sistema de interés (*system-of-interest*)	3.1.32	El sistema cuyo ciclo de vida se está analizando en el trabajo.	La aplicación móvil de Banco Pichincha (versión 15.0.0). La Banca Web, Deuna y el sistema del SRI quedan fuera, aunque la aplicación se conecte con ellos.

4. Requisitos del sistema

[Párrafo corto que presente la tabla y remita a la Figura 1.]

Tabla 2: Requisitos de la banca móvil de Banco Pichincha clasificados según [2] e [3].

ID	Enunciado	Tipo	Característica ISO/IEC 25010	Métrica	Criterio de aceptación
RQ-01	[enunciado]	[tipo]	[]	[]	[]
RQ-02	[enunciado]	[tipo]	[]	[]	[]

ID	Enunciado	Tipo	Característica ISO/IEC 25010	Métrica	Criterio de aceptación
RQ-03	[enunciado]	[tipo]	[]	[]	[]
RQ-04	[enunciado]	[tipo]	[]	[]	[]
RQ-05	[enunciado]	[tipo]	[]	[]	[]
RQ-06	[enunciado]	[tipo]	[]	[]	[]
RQ-07	[enunciado]	[tipo]	[]	[]	[]
RQ-08	[enunciado]	[tipo]	[]	[]	[]
RQ-09	[enunciado]	[tipo]	[]	[]	[]
RQ-10	[enunciado]	[tipo]	[]	[]	[]
RQ-11	[enunciado]	[tipo]	[]	[]	[]
RQ-12	[enunciado]	[tipo]	[]	[]	[]
RQ-13	[enunciado]	[tipo]	[]	[]	[]
RQ-14	[enunciado]	[tipo]	[]	[]	[]
RQ-15	[enunciado]	[tipo]	[]	[]	[]

5. Autoevaluación

[Párrafo corto: cómo se revisaron los requisitos y cuántos se corrigieron.]

Tabla 3: Requisitos corregidos en la autoevaluación.

ID	Versión anterior	Qué falló (5.2.5 / 5.2.7)	Versión corregida
[RQ-xx]	[versión anterior]	[característica o término]	[versión corregida]

6. Reflexión

[Escribir la reflexión.]

7. Declaración de uso de inteligencia artificial

Herramienta: Claude, asistente de inteligencia artificial de Anthropic.

Versión: Claude Opus 5.5.

Propósito: preparar la plantilla LaTeX del documento, el archivo README.md y el archivo de referencias; ubicar en la norma las páginas de las cláusulas 1, 3 y 5.2; [completar con el resto del uso real].

Cómo se verificó lo obtenido: [completar].

Referencias

[1] H. Washizaki, Ed., *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*. Los Alamitos, CA, USA: IEEE Computer Society, 2024.

[2] ISO/IEC/IEEE, "ISO/IEC/IEEE 29148:2018 Systems and Software Engineering — Life Cycle Processes — Requirements Engineering," International Standard, 2.ª ed., Nov. 2018, DOI: 10.1109/IEEESTD.2018.8559686.

[3] ISO/IEC, "ISO/IEC 25010:2023 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model," International Standard, 2023.